1. Which keyword is used in C to define a user-defined composite data type grouping heterogeneous elements?

(A) struct
(B) union
(C) typedef
(D) class

2. Which operator is used to access members of a structure variable directly?

(A) . (Dot operator)
(B) -> (Arrow)
(C) * (Dereference)
(D) & (Address-of)

3. Which operator is used to access structure members through a pointer to the structure?

(A) -> (Arrow operator)
(B) . (Dot operator)
(C) * (Indirection)
(D) :: (Scope)

4. The expression ptr->member is syntactically equivalent to which expression?

(A) (*ptr).member
(B) *ptr.member
(C) &ptr->member
(D) ptr.(member)

5. Which keyword creates user-defined composite types where all members share the exact same memory location?

(A) union
(B) struct
(C) enum
(D) typedef

6. What is the memory size of a union variable?

(A) Size of largest member
(B) Sum of all members
(C) Average of members
(D) Pointer size

7. What is the primary difference in memory allocation between a struct and a union?

(A) Struct sums, Union shares
(B) Struct shares, Union sums
(C) Both share memory
(D) Both sum memory

8. Which keyword creates a new, readable type alias for an existing data type in C?

(A) typedef
(B) alias
(C) type
(D) define

9. Which keyword defines a set of named integer constants in C?

(A) enum
(B) const
(C) typedef
(D) struct

10. What is the default integer value assigned to the first enumerator in an enum if unspecified?

(A) 0 (zero)
(B) 1
(C) -1
(D) Null

11. In enum { MON = 1, TUE, WED }; what integer value is assigned to WED?

(A) 3
(B) 2
(C) 1
(D) 0

12. Which preprocessor directive is used to include system standard library headers?

(A) #include <...>
(B) #include "..."
(C) #import
(D) #define

13. Which syntax includes user-defined local header files located in the current project directory?

(A) #include "..."
(B) #include <...>
(C) #use "..."
(D) #package

14. Which preprocessor directive defines a symbolic constant or parameterized macro?

(A) #define
(B) #macro
(C) #const
(D) #typedef

15. Which preprocessor directives form standard Include Guards to prevent duplicate header inclusion?

(A) #ifndef, #define, #endif
(B) #if, #else, #endif
(C) #pragma guard
(D) #include_once

16. What opaque data type defined in <stdio.h> represents a file stream pointer?

(A) FILE *
(B) FSTREAM *
(C) int fd
(D) HANDLE

17. Which standard library function opens an existing file or creates a new file?

(A) fopen()
(B) open()
(C) file_open()
(D) create_file()

18. What does fopen() return if the requested file cannot be opened or does not exist in read mode?

(A) NULL
(B) 0
(C) -1 (EOF)
(D) Throws exception

19. Which standard library function closes an open file stream and flushes its buffer?

(A) fclose()
(B) close()
(C) file_close()
(D) fflush()

20. Which file mode opens a file for reading text; fails if file does not exist?

(A) "r"
(B) "w"
(C) "a"
(D) "r+"

21. Which file mode opens a file for writing text; creates new or truncates existing file to zero length?

(A) "w"
(B) "r"
(C) "a"
(D) "w+"

22. Which file mode opens a file for appending text at the end without truncating?

(A) "a"
(B) "w"
(C) "r"
(D) "a+"

23. Which letter is appended to file mode strings to indicate binary mode across platforms?

(A) "b" (e.g. "rb", "wb")
(B) "x"
(C) "bin"
(D) "d"

24. Which function writes formatted text data to a specified file stream?

(A) fprintf()
(B) printf()
(C) fputs()
(D) fwrite()

25. Which function reads formatted data from a specified file stream?

(A) fscanf()
(B) scanf()
(C) fgets()
(D) fread()

26. Which function reads a single character from an open file stream?

(A) fgetc()
(B) getc()
(C) getchar()
(D) read_char()

27. Which function writes a single character to a specified file stream?

(A) fputc()
(B) putc()
(C) putchar()
(D) write_char()

28. Which function reads a string line safely from a file stream up to a specified character limit?

(A) fgets()
(B) gets()
(C) fscanf()
(D) fgetline()

29. Which function writes a null-terminated string to a file stream without appending extra newlines?

(A) fputs()
(B) puts()
(C) fprintf()
(D) fwrite()

30. Which standard function reads binary blocks from a file stream directly into memory?

(A) fread()
(B) read()
(C) fscanf()
(D) fgets()

31. Which standard function writes binary blocks from memory directly to a file stream?

(A) fwrite()
(B) write()
(C) fprintf()
(D) fputs()

32. How many parameters are accepted by fread() and fwrite()?

(A) 4 (ptr, size, count, fp)
(B) 2
(C) 3
(D) 5

33. Which function sets the file position indicator to a specified offset?

(A) fseek()
(B) ftell()
(C) rewind()
(D) fsetpos()

34. Which seek origin constant in fseek() indicates offsets relative to the beginning of the file?

(A) SEEK_SET
(B) SEEK_CUR
(C) SEEK_END
(D) SEEK_START

35. Which seek origin constant indicates offsets relative to current file position?

(A) SEEK_CUR
(B) SEEK_SET
(C) SEEK_END
(D) SEEK_REL

36. Which seek origin constant indicates offsets relative to the end of the file?

(A) SEEK_END
(B) SEEK_SET
(C) SEEK_CUR
(D) SEEK_TAIL

37. Which function returns the current byte offset of the file position indicator?

(A) ftell()
(B) fseek()
(C) rewind()
(D) fgetpos()

38. Which convenience function resets the file position indicator to the start of the file?

(A) rewind()
(B) freset()
(C) fseek(fp,0,0)
(D) restart()

39. Which function tests whether the End-Of-File indicator for a stream has been set?

(A) feof()
(B) ferror()
(C) eof()
(D) fis_eof()

40. Which function tests whether an error indicator has been set for a stream?

(A) ferror()
(B) feof()
(C) perror()
(D) clearerr()

41. Why is sizeof(struct { char c; int i; }) typically 8 bytes rather than 5 bytes?

(A) Structure padding for align
(B) Compiler bug
(C) int requires 7 bytes
(D) char requires 4 bytes

42. What hardware principle necessitates structure padding in modern microprocessor architectures?

(A) Memory alignment for speed
(B) Virtual RAM limits
(C) Cache size constraints
(D) Bit-field rules

43. How can compiler structure padding be completely disabled in GCC / Clang?
- **(A)** __attribute__((packed))
- **(B)** #pragma unpack
- **(C)** typedef packed
- **(D)** volatile struct

44. What preprocessor pragma controls structure member packing in MSVC and GCC?
- **(A)** #pragma pack(1)
- **(B)** #pragma align(0)
- **(C)** #pragma nopad
- **(D)** #pragma byte_align

45. Can a structure contain an instance of itself as a direct member: struct Node { struct Node next; };?
- **(A)** No (infinite size error)
- **(B)** Yes (fully valid)
- **(C)** Only if typedef'd
- **(D)** Only if static

46. How does a self-referential structure properly point to another node of its own type?
- **(A)** Pointer: struct Node *next;
- **(B)** Reference: Node &next;
- **(C)** Array: Node next[1];
- **(D)** Macro: SELF(Node)

47. What is a structure containing a pointer to its own type formally termed in computer science?
- **(A)** Self-referential structure
- **(B)** Circular structure
- **(C)** Recursive record
- **(D)** Reflective type

48. What data structures are fundamentally constructed using self-referential structures?
- **(A)** Linked lists and trees
- **(B)** Static arrays only
- **(C)** Primitive scalars
- **(D)** Bitmasks only

49. When passing a structure to a function, what does passing by value do?
- **(A)** Copies entire structure
- **(B)** Passes pointer to struct
- **(C)** Throws compile error
- **(D)** Shares single copy

50. Why is passing large structures by pointer (const Struct *) preferred over passing by value?
- **(A)** Eliminates memory copy bloat
- **(B)** Prevents stack overflow
- **(C)** Faster execution speed
- **(D)** All of the above

51. What feature allows defining structure members with exact custom bit widths?
- **(A)** Bit-Fields
- **(B)** Byte-Fields
- **(C)** Packed Enums
- **(D)** Bitwise Unions

52. In struct { unsigned int flag : 1; unsigned int code : 3; }; what is maximum value of code?
- **(A)** 7 (2^3 - 1)
- **(B)** 3
- **(C)** 8
- **(D)** 15

53. Can the address-of operator (&) be applied to a bit-field member (&s.flag)?
- **(A)** No (cannot take address)
- **(B)** Yes (returns bit pointer)
- **(C)** Only if unsigned
- **(D)** Only in GCC

54. Why cannot a programmer take the address of a bit-field member?
- **(A)** RAM addresses point to bytes
- **(B)** Bit-fields are in CPU reg
- **(C)** Compiler bug
- **(D)** Illegal syntax

55. What is a Discriminated / Tagged Union?
- **(A)** Struct with enum tag + union
- **(B)** Union of two structs
- **(C)** Bit-field union
- **(D)** Anonymous union

56. What is the purpose of the tag in a discriminated union?
- **(A)** Identifies active member
- **(B)** Calculates byte size
- **(C)** Enforces alignment
- **(D)** Cleans up memory

57. What preprocessor operator transforms a macro parameter into a quoted string literal?

(A) # (Stringizing operator)
(B) ## (Token-pasting)
(C) $ (Expansion)
(D) @ (Quote)

58. What does the macro #define PRINT_VAR(x) printf(#x " = %d\n", x) output for PRINT_VAR(count)?

(A) "count = <val>"
(B) "#x = <val>"
(C) "x = <val>"
(D) Syntax error

59. What preprocessor operator concatenates two separate tokens into a single valid token?

(A) ## (Token-pasting operator)
(B) # (Stringizing)
(C) ++ (Concat)
(D) & (Merge)

60. What does #define CONCAT(a, b) a ## b produce for CONCAT(gene_, 42)?

(A) gene_42
(B) "gene_42"
(C) gene_ ## 42
(D) gene_42()

61. Which predefined macro expands to the current source code line number as an integer?

(A) __LINE__
(B) __FILE__
(C) __DATE__
(D) __TIME__

62. Which predefined macro expands to the current source file name as a string literal?

(A) __FILE__
(B) __LINE__
(C) __STDC__
(D) __SOURCE__

63. Which function outputs a descriptive system error message based on global errno?

(A) perror()
(B) strerror()
(C) ferror()
(D) printf_error()

64. Which function in <string.h> converts an integer errno code into a descriptive error string?

(A) strerror()
(B) perror()
(C) errstring()
(D) strfault()

65. What is the purpose of fflush(FILE *stream) when applied to an output stream?

(A) Writes cached buffer to disk
(B) Clears input stream buffer
(C) Resets file pointer to 0
(D) Deletes file content

66. According to the ISO C standard, is fflush(stdin) on input streams defined behavior?

(A) Undefined behavior
(B) Well-defined (clears buffer)
(C) Implementation defined
(D) Standard ANSI C

67. In calculating file size in C, what standard sequence of fseek and ftell is used?

(A) fseek(fp,0,SEEK_END);ftell
(B) fseek(fp,0,SEEK_SET);ftell
(C) ftell(fp);fseek
(D) rewind(fp);ftell

68. What does remove("data.txt") do in standard C?

(A) Deletes file from storage
(B) Closes open file
(C) Clears file contents
(D) Moves file to trash

69. What does rename("old.txt", "new.txt") do in standard C?

(A) Renames or moves file
(B) Copies file content
(C) Creates file alias
(D) Merges files

70. In bioinformatics, what standard file format uses lines beginning with '>' for sequence headers?

(A) FASTA format
(B) PDB format
(C) GenBank format
(D) BED format

71. What file format uses 80-column fixed-width records to store 3D macromolecular structures?
- **(A)** PDB (Protein Data Bank)
- **(B)** FASTA
- **(C)** FASTQ
- **(D)** SAM / BAM

72. What 4-line repeating format stores DNA sequencing reads alongside quality scores?
- **(A)** FASTQ format
- **(B)** FASTA format
- **(C)** GFF format
- **(D)** VCF format

73. What does the 'w+' file open mode do?
- **(A)** Read and write (truncate)
- **(B)** Read only
- **(C)** Append and read
- **(D)** Binary read only

74. What does the 'r+' file open mode do?
- **(A)** Read and write (no truncate)
- **(B)** Write only
- **(C)** Truncate and write
- **(D)** Append binary

75. What does the 'a+' file open mode do?
- **(A)** Append and read from file
- **(B)** Truncate and read
- **(C)** Write only
- **(D)** Read only

76. Given struct S { char a; double b; int c; char d; }; on 64-bit alignment, what is its sizeof?
- **(A)** 24 bytes (1+7pad + 8 + 4+1+3pad)
- **(B)** 14 bytes
- **(C)** 16 bytes
- **(D)** 32 bytes

77. How can the members of struct S { char a; double b; int c; char d; } be reordered to minimize padding?
- **(A)** double b; int c; char a; char d;
- **(B)** char a; char d; int c; double b;
- **(C)** Both A and B yield 16 bytes
- **(D)** Cannot be reduced

78. When reordered as struct S { double b; int c; char a; char d; }; what is its new packed sizeof?
- **(A)** 16 bytes (saved 8 bytes!)
- **(B)** 24 bytes
- **(C)** 14 bytes
- **(D)** 12 bytes

79. Why does writing a struct directly to disk via fwrite(&s, sizeof(s), 1, fp) create portability bugs?
- **(A)** Padding and endianness differ
- **(B)** Data is corrupted
- **(C)** fwrite only writes text
- **(D)** Cannot write structs

80. What is Endianness in computer systems?
- **(A)** Byte order in multi-byte data
- **(B)** Bit alignment in words
- **(C)** File system format
- **(D)** Floating-point precision

81. In Little-Endian systems (x86/x64), which byte of a multi-byte integer is stored at lowest address?
- **(A)** Least Significant Byte (LSB)
- **(B)** Most Significant Byte (MSB)
- **(C)** Sign bit byte
- **(D)** Parity byte

82. In Big-Endian systems (network byte order), which byte is stored at lowest address?
- **(A)** Most Significant Byte (MSB)
- **(B)** Least Significant Byte (LSB)
- **(C)** Middle byte
- **(D)** Zero byte

83. How should binary file structures be designed for cross-platform portability?
- **(A)** Fixed-width types & serialization
- **(B)** Rely on default struct writes
- **(C)** Use floating-point text only
- **(D)** Use #pragma pack only

84. What functions convert 16-bit and 32-bit integers between host and network byte orders?
- **(A)** htons, htonl, ntohs, ntohl
- **(B)** hton, ntoh, bswap
- **(C)** endian_conv, byte_swap
- **(D)** int_to_net, net_to_int

85. What is a Flexible Array Member in C99?

(A) Unsized array as last member
(B) Array of pointers
(C) VLA inside struct
(D) Dynamic union

86. What is the syntax for a Flexible Array Member in C99 struct?

(A) struct Rec { int n; char seq[]; };
(B) struct Rec { int n; char seq[0]; };
(C) struct Rec { int n; char *seq; };
(D) struct Rec { int n[]; };

87. How is memory allocated for a struct containing a Flexible Array Member seq[] of size N?

(A) malloc(sizeof(struct Rec) + N)
(B) malloc(sizeof(struct Rec))
(C) malloc(N)
(D) calloc(N, sizeof(char))

88. Does sizeof(struct Rec) include the size of the Flexible Array Member seq[]?

(A) No (size is 0 / omitted)
(B) Yes (includes N bytes)
(C) Includes pointer size
(D) Undefined

89. Why was the GCC 'struct hack' (char seq[1] or seq[0]) replaced by C99 flexible array members?

(A) Codified standard behavior
(B) Avoided bounds warnings
(C) Eliminated wasted byte
(D) All of the above

90. In processing large genomic files (> 4 GB) on 32-bit systems, why does ftell() fail?

(A) 32-bit long overflows at 2GB
(B) File pointer corrupted
(C) Buffer too small
(D) Disk error

91. Which 64-bit file positioning functions handle files > 4 GB in POSIX and modern C?

(A) fseeko() and ftello() [off_t]
(B) fseek64() and ftell64()
(C) fsetpos64()
(D) Both A and B

92. What does setvbuf(fp, buffer, _IOFBF, size) allow a C program to configure?

(A) Full custom stream buffering
(B) Line buffering only
(C) Disable all I/O
(D) File encryption

93. What are the three buffering modes selectable via setvbuf() in C?

(A) _IOFBF, _IOLBF, _IONBF
(B) _FULL, _LINE, _NONE
(C) _BUFFERED, _UNBUFFERED
(D) _FAST, _NORMAL, _SLOW

94. What does _IONBF passed to setvbuf() accomplish?

(A) Disables all buffering (raw)
(B) Enables line buffering
(C) Maximizes buffer size
(D) Flushes on newline

95. In C, which standard stream is typically unbuffered (_IONBF) by default?

(A) stderr
(B) stdout
(C) stdin
(D) stdlog

96. Why is stderr unbuffered while stdout is line-buffered?

(A) Error messages appear immediately
(B) stderr is slower
(C) stderr writes to disk
(D) stdout is read-only

97. What function creates an anonymous temporary file that is automatically deleted when closed?

(A) tmpfile()
(B) tempfile()
(C) tmpnam()
(D) mktemp()

98. Why is the legacy function tmpnam() considered insecure in security-sensitive systems?

(A) Race condition (TOCTOU)
(B) Cannot write binary
(C) Buffer overflow
(D) Memory leak

99. What POSIX function securely creates a unique temporary file without race conditions?

(A) mkstemp()
(B) tmpfile()
(C) tempnam()
(D) creat()

100. What does feof(fp) return when an attempt to read PAST the end of the file occurs?

(A) Non-zero (true)
(B) 0 (false)
(C) -1 (EOF)
(D) NULL

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Structures (struct) in C, detailing syntax declaration, member access operators (. vs ->), and nested structures.

Q2. Detail Structure Padding, Data Alignment, and sizeof(struct) calculations, explaining #pragma pack(1) and __attribute__((packed)).

Q3. Compare Structures with Unions, detailing memory overlap, type punning, and memory size calculation formulas.

Q4. Explain Tagged / Discriminated Unions in C, implementing a type-safe polymorphic biological entity container.

Q5. Detail Bit-Fields in C, explaining packing syntax, member constraints, hardware register mapping, and endianness issues.

Q6. Explain typedef and enum in C, detailing type aliasing, code readability, function pointer shortcuts, and underlying integral types.

Q7. Detail the C Preprocessor pipeline: #define macros, token-pasting (##), stringizing (#), and predefined macros.

Q8. Explain File Handling in C, detailing streams (FILE*), file open modes ("r", "w", "a", "r+", "w+", "a+", "b"), and buffer flushing.

Q9. Write an ANSI C program to write and read formatted experimental assay records to a text file using fprintf() and fscanf().

Q10. Write an ANSI C program to read a multi-line FASTA genomic file using fgets() and count total base pairs.

Q11. Detail Binary File I/O in C: fread(), fwrite(), binary modes ("rb", "wb"), and structure serialization.

Q12. Explain Random Access File Handling in C, detailing fseek(), ftell(), rewind(), and SEEK_SET, SEEK_CUR, SEEK_END origin flags.

Q13. Analyze the dangerous while (!feof(fp)) anti-pattern in C file handling, explaining why feof() only triggers AFTER read failure.

Q14. Write an ANSI C program to serialize and deserialize an array of Protein Structures to a binary file with integrity verification.

Q15. Explain Flexible Array Members in C99 structures, detailing memory allocation, sizeof semantics, and the legacy struct hack.

Q16. Write an ANSI C program to copy a file byte-by-byte using fgetc() and fputc() with error checking.

Q17. Explain the role of errno, perror(), and strerror() in standard C file and system call error diagnostics.

Q18. Write an ANSI C program to parse a CSV file of clinical biochemical parameters and compute group averages.

Q19. Explain Include Guards vs #pragma once, detailing preprocessor token mechanics, multi-file builds, and compiler support.

Q20. Write an ANSI C program to implement a self-referential structure representing a Singly Linked List of DNA base nodes.

Q21. Explain stream buffering modes in C: Full Buffering (_IOFBF), Line Buffering (_IOLBF), and No Buffering (_IONBF) via setvbuf().

Q22. Write an ANSI C program to count total lines, words, and characters in a text file (equivalent to UNIX wc utility).

Q23. Explain memory-mapped file access vs standard buffered stream I/O in large-scale bioinformatics data pipelines.

Q24. Write an ANSI C program demonstrating the use of temporary files using tmpfile().

Q25. Explain the endianness problem when transmitting binary structures across network sockets or disk files with code remedies.

Q26. Write an ANSI C program to append log telemetry timestamps to an existing file using "a" open mode.

Q27. Explain anonymous unions and anonymous structures in modern C, detailing member flattening.

Q28. Write an ANSI C program to reverse lines of a file by reading lines into a dynamic array of strings.

Q29. Explain structure assignment semantics in C, detailing bitwise member copy and pointers aliasing.

Q30. Write an ANSI C program to create a binary indexed database of genomic gene records with constant-time random lookups by ID.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[D]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[D]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[D]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[C]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONSdeclaration, member access operators (. vs ->), and nested structures.

```
  gene.locus.start_position = 32315474L;
  gene.locus.end_position = 32400266L;
  gene.expression_level = 45.8;

  struct GeneAnnotation *ptr = &gene;

  printf("=== Genomic Annotation Record ===\n");
  printf("Ensembl ID: %s\n", ptr->gene_id);
  printf("Gene Symbol: %s\n", ptr->symbol);
  printf("Locus: Chr %d : %ld - %ld (Span: %ld bp)\n",
  ptr->locus.chromosome, ptr->locus.start_position, ptr->locus.end_position,
  ptr->locus.end_position - ptr->locus.start_position);
  printf("FPKM Level: %.2f\n", ptr->expression_level);

  return 0;
  }
```

>> Solution to Question 2: Detail Structure Padding, Data Alignment, and sizeof(struct) calculations, explaining #pragma pack(1) and __attribute__((packed)).

```
#include <stdio.h>

struct PaddedStruct {
char a;
double b;
int c;
};

#pragma pack(push, 1)
struct PackedStruct {
char a;
double b;
int c;
};
#pragma pack(pop)

int main(void) {
printf("Padded Struct Size: %zu bytes (Includes alignment gaps)\n", sizeof(struct
PaddedStruct));
printf("Packed Struct Size: %zu bytes (Pure sum of member sizes: 1+8+4=13)\n", sizeof(struct
PackedStruct));
return 0;
}
```

>>Solution to Question 3: Compare Structures with Unions, detailing memory overlap, type punning, and memory size calculation formulas.

4. Demonstrative ANSI C Program:

```
#include <stdio.h>

union VariantData {
int int_val;
double dbl_val;
char str_val[16];
};

int main(void) {
union VariantData u;

printf("sizeof(union VariantData): %zu bytes\n", sizeof(u));
printf("Address of u.int_val: %p\n", (void *)&u.int_val);
printf("Address of u.dbl_val: %p\n", (void *)&u.dbl_val);
printf("Address of u.str_val: %p\n", (void *)u.str_val);
printf("-> All union members share the exact same starting address!\n\n");

/* Storing integer */
u.int_val = 12345;
printf("Stored int: %d\n", u.int_val);

/* Overwriting with double */
u.dbl_val = 98.7654;
printf("Stored dbl: %.4f\n", u.dbl_val);
printf("Reading int after dbl write: %d (Overwritten/corrupted!)\n", u.int_val);

return 0;
}
```

>>Solution to Question 4: Plain Tagged / Discriminated Unions in C, implementing a type-safe polymorphic biological entity container.

```
TYPE_PROTEIN,
TYPE_CHEMICAL_BUFFER
} BiomoleculeType;

typedef struct {
BiomoleculeType type;
char sample_id[16];
union {
struct { int base_pairs; double gc_content; } dna;
struct { int amino_acids; double pI_value; } protein;
struct { double ph_level; double molarity; } buffer;
} payload;
} BiologicalSample;

void inspect_sample(const BiologicalSample *s) {
printf("Sample ID: %s | ", s->sample_id);
switch (s->type) {
case TYPE_DNA:
printf("DNA Entity: %d bp, GC: %.1f%%\n",
s->payload.dna.base_pairs, s->payload.dna.gc_content);
break;
case TYPE_PROTEIN:
printf("Protein Entity: %d residues, Isoelectric Point (pI): %.2f\n",
s->payload.protein.amino_acids, s->payload.protein.pI_value);
break;
case TYPE_CHEMICAL_BUFFER:
printf("Buffer Entity: pH %.2f at %.2f M concentration\n",
s->payload.buffer.ph_level, s->payload.buffer.molarity);
break;
}
}

int main(void) {
BiologicalSample s1, s2;

s1.type = TYPE_DNA;
strcpy(s1.sample_id, "DNA-001");
s1.payload.dna.base_pairs = 4500;
s1.payload.dna.gc_content = 52.4;

s2.type = TYPE_PROTEIN;
strcpy(s2.sample_id, "PROT-042");
s2.payload.protein.amino_acids = 380;
s2.payload.protein.pI_value = 6.85;

inspect_sample(&s1);
inspect_sample(&s2);

return 0;
}
```

>> Solution to Question 5:

[Problem Statement]: Detail Bit-Fields in C, explaining packing syntax, member constraints, hardware register mapping, and endianness issues.
[Detailed Step-by-Step Answer]:
Bit-Fields in C: Syntax, Constraints, and Systems Applications:
1. Syntax and Definition:
Bit-fields permit declaring structure members with explicit, custom bit widths, packing multiple variables into single bytes/words:

```
struct BitFieldDefinition {
type member_name : bit_width;
};
```

- Allowed Types: unsigned int, signed int, _Bool (C99).

2. Memory Conservation Example:

```
struct PacketHeader {
unsigned int version : 4; /* 4 bits (0-15) */
unsigned int header_len : 4; /* 4 bits (0-15) */
unsigned int tos : 8; /* 8 bits (0-255) */
unsigned int total_len : 16; /* 16 bits (0-65535) */
};
```

Packs all 4 fields into exactly 32 bits (4 bytes)!

3. Strict Bit-Field Constraints:
- No Address-Of Operator (&): Bit-fields do NOT have distinct byte memory addresses. Code like &hdr.version causes an immediate compile error!
- No Array of Bit-Fields: Arrays require contiguous element byte addresses.
- No sizeof on Bit-Field Members: sizeof(hdr.version) is illegal.

4. Endianness and Portability Caveat:
The ISO C standard leaves the layout order of bits within an integer word (low-order to high-order vs high-order to low-order) IMPLEMENTATION-DEFINED!
Bit-fields written on x86 (little-endian) are reversed relative to network protocols or big-endian processors.

5. Complete ANSI C Demonstration Program:

```
#include <stdio.h>

struct BioreactorStatus {
unsigned int impeller_active : 1; /* 1 bit: 0 or 1 */
unsigned int heater_active : 1; /* 1 bit: 0 or 1 */
unsigned int acid_valve_open : 1; /* 1 bit: 0 or 1 */
unsigned int base_valve_open : 1; /* 1 bit: 0 or 1 */
unsigned int sensor_fault : 1; /* 1 bit: 0 or 1 */
unsigned int alarm_severity : 3; /* 3 bits: 0 to 7 */
};

int main(void) {
struct BioreactorStatus status = {0};
status.impeller_active = 1;
status.acid_valve_open = 1;
status.alarm_severity = 5;

printf("Bioreactor packed status footprint: %zu byte(s)\n", sizeof(status));
printf("Impeller: %u | Acid Valve: %u | Alarm Severity: %u of 7\n",
status.impeller_active, status.acid_valve_open, status.alarm_severity);
```

```
return 0;
}
```

>>Solution to Question 6: Explain typedef and enum in C, detailing type aliasing, code readability, function pointer shortcuts, and underlying integral types.

- The FILE structure (defined in <stdio.h>) maintains stream state: file descriptor, read/write offset position, error flags, and I/O buffer pointers.

2. The 12 Standard File Open Modes (fopen):
Mode Read/Write? File Exists? File Missing? Initial Position
"r" Read-Only Opens file FAILS (NULL) Beginning
"w" Write-Only TRUNCATES to 0 CREATES new Beginning
"a" Append-Only Preserves content CREATES new End of File
"r+" Read/Write Preserves content FAILS (NULL) Beginning
"w+" Read/Write TRUNCATES to 0 CREATES new Beginning
"a+" Read/Append Preserves content CREATES new End for writing
Appending "b" (e.g. "rb", "wb+") disables CRLF/LF line-ending translations, mandatory for binary files!

3. Stream Buffer Flushing (fflush):
Standard C file output is buffered in user-space RAM for efficiency.
fflush(fp) forces all buffered data to be written immediately to disk.

4. Demonstrative ANSI C Program:

```
#include <stdio.h>

int main(void) {
FILE *fp = fopen("lab_log.txt", "w");
if (fp == NULL) {
perror("Failed to open file");
return 1;
}

fprintf(fp, "Experiment ID: EXP-2026-BT\n");
fprintf(fp, "Incubation Temp: 37.0 C\n");
fflush(fp); /* Ensure data is safely on disk */

fclose(fp);
printf("File created and flushed successfully.\n");
return 0;
}
```

>> Solution to Question 9:

```
double concentration;
double absorbance;
} AssayRecord;

int main(void) {
const char *filename = "assay_data.txt";
AssayRecord samples[] = {
{"SMP-01", 14.50, 0.452},
{"SMP-02", 28.90, 0.891},
{"SMP-03", 5.25, 0.165}
};
int count = sizeof(samples) / sizeof(samples[0]);

/* 1. Writing formatted text records */
FILE *fout = fopen(filename, "w");
if (!fout) { perror("Write open error"); return EXIT_FAILURE; }

for (int i = 0; i < count; i++) {
fprintf(fout, "%-10s %8.2f %8.4f\n",
samples[i].sample_id, samples[i].concentration, samples[i].absorbance);
}
fclose(fout);

/* 2. Reading formatted text records */
FILE *fin = fopen(filename, "r");
if (!fin) { perror("Read open error"); return EXIT_FAILURE; }

printf("=== Parsed Laboratory Assay Records ===\n");
char id[16];
double conc, abs_val;
while (fscanf(fin, "%15s %lf %lf", id, &conc, &abs_val) == 3) {
printf("Sample: %-10s | Conc: %6.2f ug/mL | Abs: %.4f OD\n", id, conc, abs_val);
}
fclose(fin);

return EXIT_SUCCESS;
}
```

>>Solution to Question 10: an ANSI C program to read a multi-line FASTA genomic file using fgets() and count total base pairs,

```
#include <string.h>

#define LINE_BUFFER 1024

void parse_fasta_file(const char *filename) {
FILE *fp = fopen(filename, "r");
if (!fp) {
perror("FASTA file open error");
return;
}

char line[LINE_BUFFER];
int sequence_count = 0;
long total_bases = 0;
char current_header[128] = "";

while (fgets(line, sizeof(line), fp) != NULL) {
/* Strip trailing newlines */
line[strcspn(line, "\r\n")] = '\0';

if (line[0] == '>') {
/* Header line */
if (sequence_count > 0) {
printf(" -> Base count: %ld bp\n", total_bases);
}
sequence_count++;
strncpy(current_header, line + 1, sizeof(current_header) - 1);
printf("Record %d: %s\n", sequence_count, current_header);
total_bases = 0;
} else {
/* Sequence lines */
for (int i = 0; line[i] != '\0'; i++) {
if (isalpha((unsigned char)line[i])) {
total_bases++;
}
}
}
}

if (sequence_count > 0) {
printf(" -> Base count: %ld bp\n", total_bases);
}

fclose(fp);
}

int main(void) {
/* Create demo FASTA file */
FILE *demo = fopen("sample.fasta", "w");
fprintf(demo, ">seq1 Human Insulin Exon 1\nATGCGATCGA\nTCGATCGATC\n");
fprintf(demo, ">seq2 Beta Actin Promoter\nGGCCATATAT\nAGCTAGCTA\n");
fclose(demo);

parse_fasta_file("sample.fasta");
```

```
return 0;
}
```

>>Solution to Question 11: Binary File I/O in C: fread(), fwrite(), binary modes ("rb", "wb"), and structure serialization.

```
  for (size_t i = 0; i < read_items; i++) {
  printf("Gene ID: %d | Locus: %s | MW: %.1f Da\n",
  loaded[i].gene_id, loaded[i].locus_tag, loaded[i].mw_daltons);
  }

  return 0;
  }
```

>> Solution to Question 12: Explain Random Access File Handling in C, detailing fseek(), ftell(), rewind(), and SEEK_SET, SEEK_CUR, SEEK_END origin flags.

```
/* Check file size */
fseek(fp, 0L, SEEK_END);
long total_bytes = ftell(fp);
printf("Total file size: %ld bytes (10 * %zu)\n", total_bytes, sizeof(int));

fclose(fp);
return 0;
}
```

>>Solution to Question 14: Analyze the following code snippet and identify any errors or potential issues: FILE *fp = fopen("data.txt", "r"); if (fp == NULL) { perror("Error opening file"); return 1; } int ch; while ((ch = fgetc(fp)) != EOF) { putchar(ch); } fclose(fp); Explanation: The code opens a file named "data.txt" in read mode ("r"). It checks if the file pointer fp is NULL, indicating a failure to open the file. If the file cannot be opened, it prints an error message using perror and returns 1. The code then reads characters from the file using fgetc in a while loop until the end of file (EOF) is reached. Each character read is printed to the standard output using putchar. Finally, the file is closed using fclose(fp).

[Detailed Step-by-Step Answer]:
Binary Serialization and Deserialization of Biological Entities:
1. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
int accession_id;
char symbol[8];
double molecular_weight;
int residue_count;
} ProteinRecord;

int main(void) {
const char *db_file = "proteins.bin";
ProteinRecord write_catalog[3] = {
{1001, "INS", 5808.0, 51},
{1002, "HBB", 15867.0, 146},
{1003, "MYO", 17083.0, 154}
};

/* 1. Serialize */
FILE *fout = fopen(db_file, "wb");
if (!fout) return 1;
fwrite(write_catalog, sizeof(ProteinRecord), 3, fout);
fclose(fout);

/* 2. Deserialize */
FILE *fin = fopen(db_file, "rb");
if (!fin) return 1;
ProteinRecord read_catalog[3];
size_t items_loaded = fread(read_catalog, sizeof(ProteinRecord), 3, fin);
fclose(fin);

/* 3. Verification */
printf("=== Deserialized Protein Catalog Verification ===\n");
for (size_t i = 0; i < items_loaded; i++) {
printf("ID: %d | Symbol: %-4s | MW: %8.1f Da | Length: %3d AA\n",
read_catalog[i].accession_id, read_catalog[i].symbol,
read_catalog[i].molecular_weight, read_catalog[i].residue_count);
}

return 0;
}
```

>> **Solution to Question 15:** Explain Flexible Array Members in C99 structures, detailing memory allocation, sizeof semantics, and the legacy struct hack.

incomplete array type:
struct Packet {
int length;
char payload[]; /* Flexible array member: no dimension */
};

2. Memory Allocation Pattern:
Allocate the structure size plus the desired dynamic array capacity in a single continuous malloc() call:
size_t n = 500;
struct Packet *pkt = malloc(sizeof(struct Packet) + (n * sizeof(char)));
pkt->length = n;

3. Semantics and Advantages:
- Single Allocation: Structure header and payload reside in a single contiguous heap block, improving cache locality.
- Single Free: Entire entity is released with a single free(pkt).
- sizeof(struct Packet) evaluates ONLY to the header members (payload size is omitted).

4. Contrast with Legacy 'Struct Hack':
Before C99, programmers used char payload[1]; or char payload[0];, which triggered compiler bounds warnings or relied on non-standard GCC extensions.

>>Solution to Question 16: Write an ANSI C program to copy a file byte-by-byte using fgetc() and fputc() with error checking.

```
  printf("Successfully cloned %ld bytes from %s -> %s\n", bytes_copied, src_path, dest_path);
  return 0;
  }

  int main(void) {
  /* Create source file */
  FILE *f = fopen("source_data.txt", "w");
  fprintf(f, "Gene Sequence Cloning Demonstration\nLine 2 Data\n");
  fclose(f);

  copy_file("source_data.txt", "clone_data.txt");
  return 0;
  }
```

>>Solution Question 18 ANSI C program to read a CSV file of distilled buffer volumes and compute group averages.

```
/* Create sample CSV */
FILE *init = fopen(csv_file, "w");
fprintf(init, "PatientID,Glucose_mgdL,Cholesterol_mgdL\n");
fprintf(init, "P101,95.5,180.2\n");
fprintf(init, "P102,142.0,240.5\n");
fprintf(init, "P103,110.5,195.0\n");
fclose(init);

FILE *fp = fopen(csv_file, "r");
if (!fp) return 1;

char line[128];
/* Skip header line */
if (!fgets(line, sizeof(line), fp)) { fclose(fp); return 1; }

int count = 0;
double total_glucose = 0.0, total_cholesterol = 0.0;

while (fgets(line, sizeof(line), fp) != NULL) {
char *pid = strtok(line, ",\r\n");
char *glu = strtok(NULL, ",\r\n");
char *cho = strtok(NULL, ",\r\n");

if (pid && glu && cho) {
total_glucose += atof(glu);
total_cholesterol += atof(cho);
count++;
}
}
fclose(fp);

printf("=== Clinical Biomarker Cohort Report ===\n");
printf("Total Patients Analyzed: %d\n", count);
printf("Mean Fasting Blood Glucose: %.2f mg/dL\n", total_glucose / count);
printf("Mean Total Cholesterol: %.2f mg/dL\n", total_cholesterol / count);

return 0;
}
```

>> Solution to Question 19: Explain Include Guards vs #pragma once, detailing preprocessor token mechanics, multi-file builds, and compiler support.

#endif
- Advantages: 100% ISO C standard compliant across all compilers since 1989.
- Disadvantages: Requires unique guard macro names; preprocessor must reopen and parse file lines to check guard.

3. #pragma once:
Non-standard but universally supported compiler directive placed at top of header.
- Advantages: Concise; eliminates macro naming collisions; faster compilation (compiler never re-opens the file).
- Caveats: May fail with complex symlinks or network drive aliasing.

>> Solution to Question 20: an ANSI C program to implement a self-referential structure representing a Singly Linked List of DNA base nodes.

```
}

int main(void) {
DnaNode *strand = NULL;
strand = append_base_node(strand, 'A');
strand = append_base_node(strand, 'T');
strand = append_base_node(strand, 'G');
strand = append_base_node(strand, 'C');

print_dna_list(strand);
free_dna_list(strand);
return 0;
}
```

>>Solution to Question 22 ANSI C uses full buffering (_IOFBF) for files by default, line buffering (_IOLBF) for terminals, and no buffering (_IONBF) via setvbuf().

```
FILE *fp = fopen(path, "r");
if (!fp) return 1;

long chars = 0, words = 0, lines = 0;
bool in_word = false;
int c;

while ((c = fgetc(fp)) != EOF) {
chars++;
if (c == '\n') lines++;

if (isspace(c)) {
in_word = false;
} else if (!in_word) {
in_word = true;
words++;
}
}
fclose(fp);

printf("Lines: %ld | Words: %ld | Characters: %ld\n", lines, words, chars);
return 0;
}
```

>> Solution to Question 28: ... ANSI C ... ... bioinformatics data pipelines.

```
FILE *temp = tmpfile();
if (!temp) { perror("tmpfile error"); return 1; }

fprintf(temp, "Intermediate Polymerase Chain Reaction Cycle Data\n");
rewind(temp); /* Reset to beginning */

char buffer[128];
if (fgets(buffer, sizeof(buffer), temp)) {
printf("Read from anonymous temp file: %s", buffer);
}

/* Closing automatically deletes the temporary file from disk */
fclose(temp);
printf("Temporary file automatically erased from disk storage.\n");

return 0;
}
```

>>Solution to Question 28: Diagnose and resolve temporary file creation errors in ANSI C programs on modern operating systems. Learn how tmpfile() works and when to use it over manual file handling with code remedies.

```
  if (!fp) { perror("Log append error"); return; }

  time_t raw_time = time(NULL);
  struct tm *info = localtime(&raw_time);

  char time_str[32];
  strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", info);

  fprintf(fp, "[%s] %s\n", time_str, event_message);
  fclose(fp);
  }

  int main(void) {
  const char *log = "telemetry.log";
  append_log_entry(log, "Incubator temperature stabilized at 37.0 C.");
  append_log_entry(log, "Optical density reached target setpoint 0.600 OD.");
  printf("Appended telemetry entries successfully.\n");
  return 0;
  }
```

>> Solution to Question 27: Explain anonymous unions and anonymous structures in modern C, detailing member flattening.

>>**Solution** **Question** ANSI C assignment exercises Code live relative member command operators aliasing.

2. Shallow Copy Aliasing Hazard:
If a structure contains pointers:
struct Buffer { int *data; };
struct Buffer b1, b2;
b1.data = malloc(100);
b2 = b1; /* b2.data receives the EXACT SAME MEMORY ADDRESS as b1.data! */
Both structures now alias the same heap memory block. Freeing both causes a catastrophic double-free crash!
Always perform manual deep copies for structures containing pointers.

>> Solution to Question 30: an ANSI C program to create a binary indexed database of genomic gene records with constant-time random lookups by ID.

```
  fclose(fp);
  return 0;
  }

  size_t read_count = fread(result, sizeof(GeneRecord), 1, fp);
  fclose(fp);
  return (read_count == 1 && result->id == target_id);
  }

  int main(void) {
  const char *db = "genomic_index.db";
  write_database(db);

  int lookup_id = 3;
  GeneRecord record;
  if (query_gene_by_id(db, lookup_id, &record)) {
  printf("Instantaneous O(1) Query Result for ID %d:\n", lookup_id);
  printf("Gene Name: %s\n", record.name);
  printf("GC-Content: %.2f%%\n", record.gc_content);
  } else {
  printf("Record ID %d not found.\n", lookup_id);
  }

  return 0;
  }
```
